

Using classes and modules

Jarmo Hurri

Introduction

- exploration of simplest case in object-oriented programming
 - using pre-written class (here `str`)
- using builtin modules
- IB 2.1.2

Class, instance and method

- class is a datatype
- object is instance of some class
- *method* is a function
 - defined in class (we will do this later)
 - intended to operate on either
 - instance (*instance method*)
 - class itself (*class method*)

- for example

```
greeting = 'Hi there' # instance
print(type(greeting))
print(type(str)) # class
print(type(str.upper)) # method
```

Instance method signature

- method `upper()` of class `str` returns uppercase version of string
- can be called using dot notation

```
print('abc'.upper())
```

- to see its signature and help text, evaluate

```
help(str.upper)
```

```
Help on method_descriptor:
```

```
upper(self, /) unbound builtins.str method
    Return a copy of the string converted to uppercase.
```

- what are “parameters” `self` and `/`?
- `/` is not a parameter but special symbol
 - `/` indicates end of *positional-only* parameters
 - discussed below

Instance methods: bound and unbound

- first parameter of *instance method* is object itself
 - traditional to name it `self`
- when dot notation is used, first parameter is *automatically* bound to object itself, creating *bound method*
- example

```
help('abc'.upper) # 1st parameter already bound
```

```
Help on built-in function upper:
```

```
upper() method of builtins.str instance
```

```
Return a copy of the string converted to uppercase.
```

- in this example, `str.upper` is *unbound* version of this method
 - if used, object must be passed as first parameter

```
help(str.upper)
```

```
print(str.upper('abc'))
```

Example 1

1. Take a look at the documentation of method `str.replace()`.
2. Use the method to replace all occurrences of space with
<pause>
in the string
what is this?
3. Repeat previous step, but only replace first occurrence by
using the *keyword parameter* `count`.

Example 2

1. Take a look at the documentation of method `str.count()`.
2. Write a function `last_dot_kept(s)` returning a copy of `s` with all but the last dot (`.`) replaced by the string `-dot-`.

String as sequence and iterable

- in Python, data types which can be indexed with `[]` are *sequences*
- sequences include lists, tuples and strings
- example

```
s = 'thingamabob'
print(s[3], s[1:], s[-4:-1])
```

```
n hingamabob abo
```

- string is also iterable, each iteration giving one character
- example

```
s = 'thingamabob'
for c in s:
    if 'a' <= c <= 'g': print(c, end=' ')
```

```
g a a b b
```

Example 3

1. Take a look at the documentation of method `str.find()`.
2. Use the method to write a function `find_all(s, sub)` returning a list of all indices in `s` where substring `sub` is found. The matched positions are allowed to overlap.
3. Apply your function to the case where

```
s = 'ababab'
```

and the following values of `sub`:

```
sub = 'aba'
```

```
sub = 'ab'
```

```
sub = 'b'
```

Strings are immutable

- once created, contents of string can *not* be modified
- example

```
s = 'whatever'
print(s[0]) # can be accessed
try:
    s[0] = 'W' # but not assigned
    print(s)
except TypeError:
    print('element assignment resulted in error')

s = 'W' + s[1:] # however, s can be rebound
print('rebound:', s)
```

W

element assignment resulted in error

rebound: Whatever

Correct way to build strings gradually

- immutability suggests rebinding to build from components
- slow: rebinding requires destroying and creating objects
- correct approach is to
 1. first build list of string parts (appending is cheap)
 2. then join parts together
- example ↪

```
bin_code = [0, 1, 1, 0, 0, 1] # binary code
def binary_to_morse(binary):
    if binary == 0: return 'dit'
    else: return 'dah'
morse_code = [binary_to_morse(b) for b in bin_code]
print(morse_code)
message = '-'.join(morse_code)
print(message)

['dit', 'dah', 'dah', 'dit', 'dit', 'dah']
dit-dah-dah-dit-dit-dah
```

© Jarmo Hurri

10/19

Working with strings (s and w)

operation	meaning
<code>len(s)</code>	length of string (number of chars)
<code>s + w</code>	concatenation
<code>s <= w</code>	lexicographic (dictionary) comparison
<code>for c in s: expr</code>	iterate over characters
<code>s[ind]</code>	extration by indexing or slicing
<code>s.join(iter)</code>	join elements of iter with s between
<code>s.count(w)</code>	count non-overlapping ws
<code>s.find(w)</code>	index of first found w
<code>s.replace(old, new)</code>	replace olds with new
<code>s.lower(), s.upper()</code>	to lower case, to upper case
<code>s.startswith(w)</code>	True if s starts with w
<code>s.endswith(w)</code>	True if s ends with w
<code>float(s), int(s)</code>	return float/int described by s

- some of these take optional indices to constrain to substring

© Jarmo Hurri

11/19

Modules

- *module* is file containing (reusable) Python code
- module can be taken into use with keyword `import`
- identifiers bound in module can be accessed using dot-notation

- e.g. `math.sqrt()`

- example

```
import math
# two points in xy-plane
p = (2, 0)
q = (0, 2)
print(math.dist(p, q)) # distance between points
```

2.8284271247461903

Some identifiers in module `math`

identifier	purpose
<code>sin(a)</code> , <code>cos(a)</code> , <code>tan(a)</code>	trig funcs, a in radians
<code>log(x, b)</code>	$\log_b x$
<code>ceil(x)</code>	smallest int $\geq x$
<code>floor(x)</code>	largest int $\leq x$
<code>pi</code>	π

Homework 1

Write a program which prints, on each row,

$$\alpha, \sin \alpha, \cos \alpha$$

for $\alpha = k \times \frac{\pi}{5}$, where $k = 0, 1, 2, \dots, 10$. Sample output:

```
0.0, 0.0, 1.0
0.6283185307179586, 0.5877852522924731, 0.8090169943749475
1.2566370614359172, 0.9510565162951535, 0.3090169943749475
1.8849555921538759, 0.9510565162951536, -0.30901699437494734
2.5132741228718345, 0.5877852522924732, -0.8090169943749473
3.141592653589793, 1.2246467991473532e-16, -1.0
3.7699111843077517, -0.587785252292473, -0.8090169943749476
4.39822971502571, -0.9510565162951535, -0.30901699437494756
5.026548245743669, -0.9510565162951536, 0.30901699437494723
5.654866776461628, -0.5877852522924734, 0.8090169943749473
6.283185307179586, -2.4492935982947064e-16, 1.0
```

© Jarmo Hurri

14/19

Homework 2

The Finnish social security number (SSN) is a string `ddmmyycnnh` which specifies the date of birth of the person: day (`dd`), month (`mm`), last two digits of the year (`yy`) and century `c`:

c	+	-	A
century	1800	1900	2000

Characters `nnh` are not relevant for this exercise.

Write a function `date_of_birth(ssn)` whose parameter is an SSN, and whose return value is a triple of integers (`y`, `m`, `d`) where `y` is the year of birth, `m` the month and `d` the day. You may assume that the argument is a valid SSN. Example:

```
>>> print(date_of_birth('140598+abcd'))
(1898, 5, 14)
```

© Jarmo Hurri

15/19

Homework 3

Write a function `dashify_substring(s, sub)` which returns a copy of `s` with the first occurrence of `sub` surrounded by dashes (-). The table below lists return values for some representative parameter values.

s	sub	dashify_substring(s, sub)
foo	o	f-o-o
foobar	oba	fo-oba-r
foobar	f	-f-oobar
foobar	bar	foo-bar-
foobar	cat	foobar
foobar	''	foobar

[Hint. `str.replace()`.]

Homework 4

Write a function `file_type(s)` returning the substring after the last dot ('.') in `s`. Take a look at the documentation of `str.rfind()` before designing a solution. The table below lists return values for some representative parameter values ('' indicates empty string).

s	file_type(s)
foo.doc	doc
foo.docx	docx
foo.bar.docx	docx
''	''
foo	''
foo.	''

Homework 5

With no arguments, the call `s.split()` returns a list of substrings of `s` where the substrings are separated by whitespace (space, tabular or return). Python also supports list unpacking. So:

```
[a_str, op, b_str] = '-11.6 + 28'.split()
print(f'a_str={a_str}, op={op}, b_str={b_str}')
```

```
a_str=-11.6, op=+, b_str=28
```

Write a calculator evaluating decimal number operations of the form `a + b`, `a - b` and `a * b` until the user provides an empty input. Example run:

```
my-calc: 11 - 22.3
-11.3
my-calc: -9.11 * 8.41
-76.6151
```

Homework 6

Write a function `max_char_rep(s)` returning the length of the longest substring of repetitive characters. The table below lists return values for some representative parameter values.

s	max_char_rep(s)
abcd	1
abbbcd	3
abbbcd	5
''	0